

Praktikum 12: VueJS Komponen dan Routing (Single Page Application)

Tujuan

1. Mahasiswa mampu memahami konsep komponen pada Framework VueJS.
2. Mahasiswa mampu memahami konsep Client-Side Routing untuk membangun *Single Page Application* (SPA).
3. Mahasiswa mampu mengimplementasikan komponen dan routing menggunakan Vue Router berbasis CDN pada aplikasi Frontend API yang telah dibuat.

Instruksi Praktikum

1. Persiapkan text editor misalnya VSCode.
2. Buka kembali folder dengan nama lab8_vuejs pada docroot webserver (htdocs) yang telah dibuat pada praktikum sebelumnya.
3. Ikuti langkah-langkah praktikum yang akan dijelaskan berikutnya.

Apa itu Vue Components dan Vue Router?

Vue Components adalah elemen UI modular yang dapat digunakan kembali (*reusable*). Dengan komponen, kita dapat memecah antarmuka aplikasi menjadi bagian-bagian terisolasi seperti Header, Footer, Sidebar, atau daftar data khusus, sehingga kode menjadi lebih bersih dan mudah dikelola.

Vue Router adalah library resmi untuk VueJS yang menangani pemindahan halaman di sisi klien (*Client-Side Routing*). Dalam aplikasi web tradisional, setiap kali tautan diklik, browser akan memuat ulang (refresh) seluruh halaman dari server. Dengan Vue Router, kita dapat beralih dari satu tampilan ke tampilan lain tanpa memuat ulang browser, menciptakan pengalaman pengguna yang sangat cepat yang dikenal sebagai **Single Page Application (SPA)**.

Langkah-langkah Praktikum

Persiapan

Pada praktikum ini, kita akan meningkatkan struktur project sebelumnya dengan menambahkan pustaka **Vue Router** menggunakan CDN.

Buka kembali file index.html dan tambahkan library Vue Router di dalam tag <head> setelah library VueJS dan Axios:

HTML

```
<script src="https://unpkg.com/vue-router@4/dist/vue-router.global.js"></script>
```

Struktur Direktori Baru

Untuk menjaga modularitas, sesuaikan atau pecah file JavaScript kita menjadi beberapa berkas komponen di dalam folder **assets/js/components/**. Ubah struktur berkas menjadi seperti berikut:

Lab11_vuejs

```
| index.html
└── assets
    ├── css
    │   └── style.css
    └── js
        ├── app.js
        └── components
            ├── Home.js
            └── Artikel.js
```

1. Membuat File Komponen Halaman Utama (assets/js/components/Home.js)

Buat file baru bernama **Home.js** untuk menampilkan halaman beranda/selamat datang.

JavaScript

```
const Home = {
  template: `
    <div class="home-container">
      <h2>Selamat Datang di Portal Admin Artikel</h2>
      <p>Gunakan menu navigasi di atas untuk mengelola data artikel secara real-time
      memanfaatkan RESTful API CodeIgniter 4 dan VueJS.</p>
    </div>
  `,
};
```

2. Memindahkan Kode Fitur Artikel ke Komponen (assets/js/components/Artikel.js)

Pindahkan logika CRUD artikel dari berkas app.js lama ke dalam komponen terisolasi bernama Artikel.js.

JavaScript

```
const Artikel = {
  template: `
    <div>
      <h2>Manajemen Data Artikel</h2>
      <button id="btn-tambah" @click="tambah">Tambah Data</button>

      <div class="modal" v-if="showForm">
        <div class="modal-content">
          <span class="close" @click="showForm = false">x</span>
          <form id="form-data" @submit.prevent="saveData">
            <h3>{{ formTitle }}</h3>
          </div>
        </div>
      </div>
    </div>
  `
```

```

        <input type="text" v-model="formData.judul" placeholder="Judul Artikel"
required>
    </div>
    <div>
        <textarea v-model="formData.isi" rows="6" placeholder="Isi Artikel"
required></textarea>
    </div>
    <div>
        <select v-model="formData.status">
            <option v-for="option in statusOptions" :value="option.value">
                {{ option.text }}
            </option>
        </select>
    </div>
    <input type="hidden" v-model="formData.id">
    <button type="submit" id="btnSimpan">Simpan</button>
    <button type="button" @click="showForm = false">Batal</button>
</form>
</div>
</div>

<table>
    <thead>
        <tr>
            <th>ID</th>
            <th>Judul</th>
            <th>Status</th>
            <th>Aksi</th>
        </tr>
    </thead>
    <tbody>
        <tr v-for="(row, index) in artikel" :key="row.id">
            <td class="center-text">{{ row.id }}</td>
            <td>{{ row.judul }}</td>
            <td>{{ statusText(row.status) }}</td>
            <td class="center-text">
                <a href="#" @click.prevent="edit(row)">Edit</a>
                <a href="#" @click.prevent="hapus(index, row.id)">Hapus</a>
            </td>
        </tr>
    </tbody>
</table>
</div>

```

```
,
data() {
  return {
    artikel: [],
    formData: { id: null, judul: "", isi: "", status: 0 },
    showForm: false,
    formTitle: 'Tambah Data',
    statusOptions: [
      { text: 'Draft', value: 0 },
      { text: 'Publish', value: 1 }
    ]
  }
},
mounted() {
  this.loadData();
},
methods: {
  loadData() {
    axios.get(apiUrl + '/post')
      .then(response => {
        this.artikel = response.data.artikel;
      })
      .catch(error => console.log(error));
  },
  tambah() {
    this.showForm = true;
    this.formTitle = 'Tambah Data';
    this.formData = { id: null, judul: "", isi: "", status: 0 };
  },
  edit(data) {
    this.showForm = true;
    this.formTitle = 'Ubah Data';
    this.formData = { id: data.id, judul: data.judul, isi: data.isi, status: data.status };
  },
  hapus(index, id) {
    if (confirm('Yakin menghapus data?')) {
      axios.delete(apiUrl + '/post/' + id)
        .then(response => {
          this.artikel.splice(index, 1);
        })
        .catch(error => console.log(error));
    }
  },
},
```

```
saveData() {
  if (this.formData.id) {
    axios.put(apiUrl + '/post/' + this.formData.id, this.formData)
      .then(response => { this.loadData(); })
      .catch(error => console.log(error));
  } else {
    axios.post(apiUrl + '/post', this.formData)
      .then(response => { this.loadData(); })
      .catch(error => console.log(error));
  }
  this.formData = { id: null, judul: '', isi: '', status: 0 };
  this.showForm = false;
},
statusText(status) {
  if (!status) return 'Draft';
  return status == 1 ? 'Publish' : 'Draft';
}
};
```

3. Mengonfigurasi Vue Router pada assets/js/app.js

Edit file **app.js** utama Anda untuk mendaftarkan rute internal, komponen, dan melakukan *mounting* aplikasi.

JavaScript

```
const { createApp } = Vue;
const { createRouter, createWebHashHistory } = VueRouter;

// Tentukan lokasi API REST End Point sesuai project CI4 Anda
const apiUrl = 'http://localhost/labci4/public';

// 1. Definisikan mapping rute URL ke Komponen
const routes = [
  { path: '/', component: Home },
  { path: '/artikel', component: Artikel }
];

// 2. Buat instance router
const router = createRouter({
  history: createWebHashHistory(),
  routes
});
```

```
// 3. Inisialisasi Aplikasi Vue dan gunakan Router
const app = createApp({});
app.use(router);
app.mount('#app');
```

4. Memodifikasi Master Layout pada index.html

Sesuaikan isi file index.html agar menyediakan menu navigasi menggunakan <router-link> dan tempat penampung halaman dinamis menggunakan <router-view>.

HTML

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>SPA Frontend VueJS & Vue Router</title>
  <script src="https://unpkg.com/vue@3/dist/vue.global.js"></script>
  <script src="https://unpkg.com/vue-router@4/dist/vue-router.global.js"></script>
  <script src="https://unpkg.com/axios/dist/axios.min.js"></script>
  <link rel="stylesheet" href="assets/css/style.css">
</head>
<body>
  <div id="app">
    <header>
      <h1>Aplikasi Panel Single Page (SPA)</h1>
      <nav class="nav-menu">
        <router-link to="/">Beranda</router-link> |
        <router-link to="/artikel">Kelola Artikel</router-link>
      </nav>
    </header>

    <main style="margin-top: 20px;">
      <router-view></router-view>
    </main>
  </div>

  <script src="assets/js/components/Home.js"></script>
  <script src="assets/js/components/Artikel.js"></script>
  <script src="assets/js/app.js"></script>
</body>
</html>
```

5. Tambahkan CSS Menarik pada `assets/css/style.css`

Tambahkan rule navigasi ini pada file CSS Anda agar tampilan menu terlihat rapi:

CSS

```
.nav-menu {
  padding: 10px;
  background: #eff1ff;
  border-radius: 5px;
  margin-bottom: 15px;
}
.nav-menu a {
  text-decoration: none;
  color: #3152d6;
  font-weight: bold;
  padding: 5px 10px;
}
/* Style otomatis saat route aktif */
.router-link-exact-active {
  background-color: #3152d6;
  color: #ffffff !important;
  border-radius: 3px;
}
.home-container {
  padding: 20px;
  border: 1px solid #eff1ff;
  background: #fafafa;
}
```

Pertanyaan dan Tugas

1. Selesaikan semua langkah praktikum di atas.
2. Tambahkan satu rute baru (/about) beserta komponen About.js baru yang berisi profil singkat Anda (Nama, NIM, Kelas, dan Foto/Avatar). Masukkan tautan rutenya ke dalam menu navigasi atas pada index.html.
3. Lakukan pengujian perpindahan halaman menu (Beranda, Kelola Artikel, dan About) dan pastikan browser tidak melakukan *hard-reload* (SPA bekerja).

Laporan Praktikum

1. Melanjutkan praktikum sebelumnya pada repository dengan nama Lab11Web_VueJS atau buat baru jika diperlukan.
2. Kerjakan semua latihan dan tugas modifikasi yang diberikan sesuai urutannya.
3. Ambil screenshot setiap perubahan langkah dan hasil running aplikasinya di browser.
4. Update berkas README.md dan tuliskan penjelasan lengkap dari setiap langkah

praktikum beserta bukti screenshotnya.

5. Lakukan commit dan push hasil kerja ke repository GitHub masing-masing.
6. Kirimkan URL repository GitHub Anda pada platform e-learning ecampus.